



Aide-mémoire rapide - Gestion de Truffière



Démarrage rapide

bash

Installation automatique (Linux/Mac)

bash setup.sh

OU Manuel

docker-compose up --build



URLs d'accès

- **Application web** : <http://localhost:3000>
- **API Backend** : <http://localhost:3001/api>
- **Base de données** : localhost:5432



Commandes Docker essentielles

bash

Démarrer l'application

docker-compose up *# En mode interactif*

docker-compose up -d *# En arrière-plan*

docker-compose up --build *# Reconstruire et démarrer*

Arrêter l'application

docker-compose down *# Arrêter les services*

docker-compose down -v *# + Supprimer les volumes (⚠ EFFACE LES DONNÉES)*

Gérer les services

docker-compose ps *# Voir l'état des services*

docker-compose restart backend *# Redémarrer un service*

docker-compose stop *# Arrêter sans supprimer*

docker-compose start *# Redémarrer après stop*

Logs

docker-compose logs *# Tous les logs*

docker-compose logs -f *# Suivre les logs en temps réel*

docker-compose logs -f backend *# Logs d'un service spécifique*

docker-compose logs --tail=100 *# 100 dernières lignes*



Base de données PostgreSQL

```
bash
```

```
# Accéder à PostgreSQL
```

```
docker exec -it truffiere_db psql -U truffiere_user -d truffiere
```

```
# Commandes psql utiles
```

```
\dt          # Lister toutes les tables
```

```
\d+ parcelles # Structure de la table parcelles
```

```
\l           # Lister les bases de données
```

```
\du          # Lister les utilisateurs
```

```
\q           # Quitter
```

```
# Requêtes SQL exemples
```

```
SELECT * FROM parcelles;
```

```
SELECT * FROM arbres WHERE etat = 'Bon';
```

```
SELECT COUNT(*) FROM recoltes;
```

Sauvegarder / Restaurer

```
bash
```

```
# Sauvegarde
```

```
docker exec truffiere_db pg_dump -U truffiere_user truffiere > backup_$(date +%Y%m%d).sql
```

```
# Restauration
```

```
docker exec -i truffiere_db psql -U truffiere_user truffiere < backup_20241228.sql
```

```
# Export CSV d'une table
```

```
docker exec truffiere_db psql -U truffiere_user -d truffiere \  
-c "COPY recoltes TO STDOUT WITH CSV HEADER" > recoltes.csv
```

Développement

Modifier le code

Backend : Les changements dans `backend/server.js` sont automatiquement détectés (nodemon) **Frontend** : Les changements React sont automatiquement rechargés (Hot Reload)

Ajouter une dépendance npm

```
bash
```

```
# Backend
```

```
docker-compose exec backend npm install nom-du-package
```

```
# Puis reconstruire : docker-compose up --build backend
```

```
# Frontend
```

```
docker-compose exec frontend npm install nom-du-package
```

```
# Puis reconstruire : docker-compose up --build frontend
```

Réinitialiser complètement

```
bash
```

```
# Tout supprimer et recommencer
```

```
docker-compose down -v
```

```
rm -rf backend/node_modules frontend/node_modules
```

```
docker-compose up --build
```



API Endpoints disponibles

Parcelles

- `GET /api/parcelles` - Liste des parcelles
- `GET /api/parcelles/:id` - Détail d'une parcelle
- `POST /api/parcelles` - Créer une parcelle
- `PUT /api/parcelles/:id` - Modifier une parcelle
- `DELETE /api/parcelles/:id` - Supprimer une parcelle

Arbres

- `GET /api/arbres?parcelle_id=X` - Liste des arbres
- `POST /api/arbres` - Créer un arbre

Interventions

- `GET /api/interventions?debut=YYYY-MM-DD&fin=YYYY-MM-DD`
- `POST /api/interventions`
- `GET /api/types-intervention` - Types disponibles

Récoltes

- GET /api/recoltes?annee=YYYY
- POST /api/recoltes

Clients

- GET /api/clients
- POST /api/clients

Ventes

- GET /api/ventes
- POST /api/ventes

Statistiques

- GET /api/stats/production-parcelle
- GET /api/stats/production-arbre
- GET /api/stats/ventes

Historique

- GET /api/historique?table_name=X&record_id=Y&limit=50

Test

- GET /api/health - Vérifier que l'API fonctionne



Tests API avec curl

```
bash
```

```
# Vérifier que l'API fonctionne
```

```
curl http://localhost:3001/api/health
```

```
# Récupérer toutes les parcelles
```

```
curl http://localhost:3001/api/parcelles
```

```
# Créer une nouvelle parcelle
```

```
curl -X POST http://localhost:3001/api/parcelles \  
-H "Content-Type: application/json" \  
-d '{  
  "nom": "Ma Parcelle",  
  "surface_ha": 2.5,  
  "type_sol": "Calcaire",  
  "ph_sol": 7.9,  
  "exposition": "Sud"  
}
```

```
# Modifier une parcelle
```

```
curl -X PUT http://localhost:3001/api/parcelles/1 \  
-H "Content-Type: application/json" \  
-d '{"nom": "Parcelle Modifiée"}'
```

```
# Supprimer une parcelle
```

```
curl -X DELETE http://localhost:3001/api/parcelles/1
```



Problèmes fréquents

Port déjà utilisé

```
bash
```

```
# Modifier les ports dans docker-compose.yml
```

```
ports:
```

```
- "3002:3001" # Backend sur port 3002 au lieu de 3001
```

Erreur "Cannot connect to Docker daemon"

```
bash
```

```
# Démarrer Docker Desktop ou le daemon Docker
```

```
sudo systemctl start docker # Linux
```

Page blanche / Erreur CORS

```
bash
```

```
# Vérifier que le backend répond
```

```
curl http://localhost:3001/api/health
```

```
# Vérifier les logs
```

```
docker-compose logs -f backend
```

Base de données non initialisée

```
bash
```

```
# Recréer complètement
```

```
docker-compose down -v
```

```
docker-compose up --build
```

Module not found

```
bash
```

```
# Supprimer node_modules et réinstaller
```

```
docker-compose down
```

```
rm -rf backend/node_modules frontend/node_modules
```

```
docker-compose build --no-cache
```

```
docker-compose up
```



Structure de la base de données

Tables principales :

- `parcelles` - Informations sur les parcelles
- `arbres` - Inventaire des arbres truffiers
- `types_intervention` - Catalogue des types d'interventions
- `interventions` - Planning et historique des travaux
- `recoltes` - Enregistrement des récoltes
- `clients` - Gestion des clients
- `ventes` - Suivi des ventes
- `historique` - Audit trail automatique

Vues statistiques :

- stats_production_parcelle
- stats_production_arbre
- stats_ventes

🔑 Identifiants par défaut

PostgreSQL :

- Base : truffiere
- User : truffiere_user
- Password : truffiere_pass_2024
- Port : 5432

⚠️ À changer en production !

🗡️ Navigation dans l'application

1. 📊 **Tableau de bord** - Vue d'ensemble
2. 📖 **Parcelles** - Gestion des terrains
3. 🌳 **Arbres** - Inventaire des arbres
4. 🛠️ **Interventions** - Planning des travaux
5. 🍄 **Récoltes** - Suivi de production
6. 👤 **Clients** - Fichier clients
7. 💰 **Ventes** - Gestion commerciale
8. 📈 **Statistiques** - Analyses et rapports

🚦 Vérification de santé

```
bash
```

```
# Backend
```

```
curl http://localhost:3001/api/health
```

```
# Frontend
```

```
curl http://localhost:3000
```

```
# Base de données
```

```
docker exec truffiere_db pg_isready -U truffiere_user
```

```
# Tous les services
```

```
docker-compose ps
```



Fichiers de documentation

- `README.md` - Documentation complète
- `INSTRUCTIONS_DEPLOYMENT.md` - Guide de déploiement détaillé
- `QUICK_REFERENCE.md` - Ce fichier



Astuces

Voir les requêtes SQL en temps réel

bash

Dans PostgreSQL

```
docker exec -it truffiere_db psql -U truffiere_user -d truffiere
ALTER DATABASE truffiere SET log_statement = 'all';
```

Importer des données CSV

bash

```
docker exec -i truffiere_db psql -U truffiere_user -d truffiere << EOF
COPY parcelles(nom, surface_ha, type_sol, ph_sol)
FROM '/path/to/file.csv'
DELIMITER ','
CSV HEADER;
EOF
```

Optimiser les performances

bash

Analyser les requêtes lentes

```
docker exec truffiere_db psql -U truffiere_user -d truffiere \
-c "SELECT query, calls, total_time FROM pg_stat_statements ORDER BY total_time DESC LIMIT 10;"
```



Prochaines étapes suggérées

1. ☒ Remplacer les données de démo par vos vraies données
2. ☒ Implémenter l'authentification utilisateur
3. ☒ Ajouter la cartographie interactive (Leaflet)
4. ☒ Créer les exports PDF des rapports
5. ☒ Ajouter des graphiques avancés (Recharts)
6. ☒ Implémenter l'upload de photos
7. ☒ Configurer les sauvegardes automatiques
8. ☒ Préparer le déploiement en production

Dernière mise à jour : Décembre 2024

Version : 1.0.0